

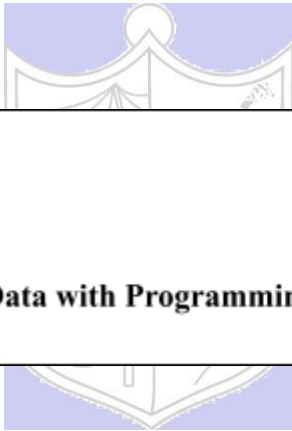


SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27



Experiment No. : 2

Title: Understanding Data with Programming Exercise



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/AIDS/SYBTECH /SEM-III/FDS/2026-27

Batch: C1

Roll No.: 16014225013

Experiment No.: 2

Aim: Understanding of the Data Handling technique with Programming exercise

Resources needed: Python 3.x environment (Jupyter Notebook / Google Colab etc.), numpy and pandas library

Theory

There are several programming languages that are commonly used for data handling, analysis, and manipulation tasks. Some of the most popular data handling programming languages are Python, R, Scala, Matlab, Julia, and SQL. In this practical session, we will handle Python-based data handling commands. Let us see features of Python.

Python

Python is one of the most widely used programming languages for data handling and analysis. It has rich libraries like Pandas, NumPy, Matplotlib, Seaborn, and SciPy that offer comprehensive support for data manipulation, numerical computing, and data visualization. Python's readability and user-friendly syntax make it a top choice for data scientists, researchers, and analysts. In this session, we will learn some important Python libraries, namely NumPy and Pandas.

NumPy

NumPy, which stands for "Numerical Python," is a fundamental package for numerical computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with an extensive collection of mathematical functions to operate on these arrays efficiently. NumPy is the backbone of many other Python libraries for scientific computing and data analysis.

Key features of NumPy:

- **N-dimensional array object (numpy.ndarray):** Efficient data structure for storing and manipulating large datasets, making it suitable for numerical computations and data analysis.
- **Broadcasting:** NumPy automatically broadcasts arrays with different shapes for element-wise operations, reducing the need for explicit loops.
- **Mathematical Functions:** NumPy offers a wide range of mathematical functions like trigonometric, exponential, statistical, and linear algebra functions, among others.
- **Indexing and Slicing:** NumPy supports advanced indexing techniques to access specific elements or subsets of data quickly.
- **Integration with C/C++:** NumPy is written in C and Python, which provides the advantage of speed and efficiency in numerical computations.

Pandas

Pandas is an open-source data manipulation library built on top of NumPy. It provides high-level data structures and functions to make working with structured data easy and intuitive. The two primary data structures in Pandas are the **Series** and **DataFrame**.

Key features of Pandas:

- **Series:** A one-dimensional labeled array, similar to a NumPy array but with additional indexing capabilities, making it more versatile for data analysis.

- **DataFrame:** A two-dimensional labeled data structure with rows and columns, resembling a spreadsheet or SQL table, which is the central data structure for data analysis in Pandas.
- **Data Import/Export:** Pandas can read and write data from various file formats like CSV, Excel, SQL databases, JSON, and HTML tables.
- **Data Exploration:** Pandas provides functions to explore, clean, and preprocess data, such as handling missing values, filtering, grouping, and aggregation.
- **Time-Series Analysis:** Pandas has robust support for handling time-series data, making it popular for financial and temporal data analysis.
- **Data Visualization:** Although not a visualization library, Pandas integrates well with other visualization libraries like Matplotlib and Seaborn, allowing easy creation of basic visualizations directly from DataFrames and Series.

Together, NumPy and Pandas form a powerful combination for data analysis and data manipulation tasks in Python. NumPy handles the underlying numerical computations and efficient data storage, while Pandas provides high-level structures and functions that simplify data handling and analysis tasks.

Exercise: Data Handling Commands with Python

Both Pandas and NumPy offer a wide range of commands and functions for data analysis. Here are some essential commands from each library commonly used in data analysis:

1. Pandas

Creating DataFrames

- `pd.DataFrame(data, columns=['col1', 'col2', ...])`: Creates a DataFrame from data (a dictionary, list of lists, or a NumPy array) with specified column names.

Data Exploration

- `df.head(n)`: Returns the first n rows of the DataFrame.
- `df.info()`: Provides information about the DataFrame, including data types and non-null counts.
- `df.describe()`: Generates summary statistics for numerical columns.
- `df.shape`: Returns the dimensions of the DataFrame (rows, columns).
- `df.columns`: Returns the column names.

Data Selection and Filtering

- `df['column_name']`: Selects a specific column from the DataFrame.
- `df.loc[row_index, 'column_name']`: Accesses specific data using labels for both rows and columns.
- `df.iloc[row_index, col_index]`: Accesses specific data using integer-based indexing.
- `df[df['column_name'] > value]`: Filters data based on a condition.

Data Aggregation

- `df.groupby('column_name').agg({'other_column': 'mean'})`: Grouping and aggregation based on specific columns.
- `df['column_name'].value_counts()`: Calculates the frequency of unique values in a column.

Data Cleaning

- `df.isnull()`: Checks for missing values in the DataFrame.
- `df.dropna()`: Drops rows with missing values.
- `df.fillna(value)`: Fills missing values with a specific value.

Data Visualization

- `df['column_name'].plot(kind='hist')`: Creates a histogram for a specific column.

- `df.plot(x='x_column', y='y_column', kind='scatter')`: Creates a scatter plot.

2. NumPy

Creating Arrays

- `np.array([1, 2, 3])`: Creates a 1-dimensional NumPy array.
- `np.zeros((m, n))`: Creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))`: Creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)`: Creates an array with values from start to stop (exclusive) with a given step size.

Array Operations

- `np.mean(arr)`: Calculates the mean of the array.
- `np.median(arr)`: Calculates the median of the array.
- `np.sum(arr)`: Calculates the sum of the array elements.
- `np.std(arr)`: Calculates the standard deviation of the array.
- `np.unique(arr)`: Returns the unique elements of the array.

Array Indexing and Slicing

- `arr[index]`: Accesses specific elements of the array using indexing.
- `arr[start:stop]`: Slices the array from start to stop (exclusive).

Array Broadcasting

- NumPy automatically broadcasts arrays with different shapes for element-wise operations.

Mathematical Functions

- `np.sin(arr)`, `np.cos(arr)`, `np.exp(arr)`, etc.: Various mathematical functions that operate element-wise on the array

Activity:

- Read dataset related to your chosen AI use case using Numpy and Pandas
- Perform in detail data analysis of this dataset
- Get information and description of dataset.
- See if any null values are present. Display count of null values.
- Choose appropriate technique to handle missing values.(imputation with use of inplace)
- Use sorting of data in dataframe to display topmost 5 or 8 records based on one or more column values(conditional filtering)
- Get frequency listing of any one relevant column(2 cases)
- Sorting of rows and columns,(implicit and explicit indexing)
- Accessing particular row based on certain condition and displaying only one or few columns from it.(3 cases with compound conditions)
- Minimum and maximum values related analysis
- Use of group by on one or more columns(2 cases)
- Adding new column to existing dataframe and populating same using existing columns data.
- Use of appropriate aggregate functions with groupby.(2 cases)

- Selection on particular groups based on name or condition
- Find correlation between any two columns values.
- Try transformation(normalization using any technique) on data set
- Joining, merging and concatenation of data in dataframe.

Write down observation for your dataset for each of above listed task of analysis.

Results:

1. Read dataset related to your chosen AI use case using Numpy and Pandas
2. Perform in detail data analysis of this dataset
3. Get information and description of dataset.

Program

```
import numpy as np
import pandas as pd

df = pd.read_csv('delhi-weather-aqi-2025.csv')
df['datetime'] = pd.to_datetime(df['date_ist'] + ' ' + df['time_ist'],
format='%d/%m/%Y %H:%M')
print(df.shape)
print(df.head())
print(df.tail())
print(df.shape)
print(df.columns.tolist())
print(df.dtypes)

df.info()
df.describe()
```

Output

```
(52560, 17)
  date Ist time Ist location lat lon temp_c humidity \
*** 0 01/01/2025 0:00 Anand Vihar 28.6469 77.316 8.1 100
    1 01/01/2025 1:00 Anand Vihar 28.6469 77.316 7.7 100
    2 01/01/2025 2:00 Anand Vihar 28.6469 77.316 7.5 100
    3 01/01/2025 3:00 Anand Vihar 28.6469 77.316 7.8 99
    4 01/01/2025 4:00 Anand Vihar 28.6469 77.316 7.3 100

  pressure_mb windspeed_kph condition_text description aqi_index pm2_5 \
0 995.4 2.9 Mainly clear WMO Code 1 197 185.8
1 994.7 3.2 Overcast WMO Code 3 198 174.6
2 994.3 4.5 Overcast WMO Code 3 199 164.4
3 994.1 6.0 Overcast WMO Code 3 200 156.5
4 993.8 6.8 Overcast WMO Code 3 200 149.5

  pm10 co no2 datetime
0 188.6 1907 56.7 2025-01-01 00:00:00
1 177.4 1669 44.8 2025-01-01 01:00:00
2 166.7 1493 34.6 2025-01-01 02:00:00
3 158.8 1401 26.7 2025-01-01 03:00:00
4 151.8 1372 20.6 2025-01-01 04:00:00

  date Ist time Ist location lat lon temp_c humidity \
52555 31/12/2025 19:00 IGI Airport 28.5562 77.1 15.0 88
52556 31/12/2025 20:00 IGI Airport 28.5562 77.1 14.4 90
52557 31/12/2025 21:00 IGI Airport 28.5562 77.1 13.8 92
52558 31/12/2025 22:00 IGI Airport 28.5562 77.1 13.4 94
52559 31/12/2025 23:00 IGI Airport 28.5562 77.1 13.3 95

  pressure_mb windspeed_kph condition_text description aqi_index \
52555 984.3 7.5 Mainly clear WMO Code 1 209
52556 984.7 6.6 Partly cloudy WMO Code 2 209
52557 985.1 5.8 Partly cloudy WMO Code 2 209
52558 985.1 6.0 Overcast WMO Code 3 209
52559 985.1 6.5 Partly cloudy WMO Code 2 208

  pm2_5 pm10 co no2 datetime
52555 148.5 149.0 1517 109.6 2025-12-31 19:00:00
52556 153.7 154.1 1637 112.5 2025-12-31 20:00:00
```

RangeIndex: 52560 entries, 0 to 52559

Data columns (total 17 columns):

```
# Column Non-Null Count Dtype
---
0 date Ist 52560 non-null object
1 time Ist 52560 non-null object
2 location 52560 non-null object
3 lat 52560 non-null float64
4 lon 52560 non-null float64
5 temp_c 52560 non-null float64
6 humidity 52560 non-null int64
7 pressure_mb 52560 non-null float64
8 windspeed_kph 52560 non-null float64
9 condition_text 52560 non-null object
10 description 52560 non-null object
11 aqi_index 52560 non-null int64
12 pm2_5 52560 non-null float64
13 pm10 52560 non-null float64
14 co 52560 non-null int64
15 no2 52560 non-null float64
16 datetime 52560 non-null datetime64[ns]
dtypes: datetime64[ns](1), float64(8), int64(3), object(5)
memory usage: 6.8+ MB
```

dtypes: datetime[1], float[8], int64[3], object[3]
memory usage: 6.8+ MB

	lat	lon	temp_c	humidity	pressure_mb	windspeed_kph	aqi_index	pm2_5	pm10	co	no2	datetime
count	52560.000000	52560.000000	52560.000000	52560.000000	52560.000000	52560.000000	52560.000000	52560.000000	52560.000000	52560.000000	52560.000000	52560
mean	28.608850	77.174567	25.024406	62.285940	983.162635	6.565820	287.364212	89.542432	300.462700	863.844273	35.442757	2025-07-02 11:30:00
min	28.527300	77.049400	5.700000	5.000000	966.100000	0.000000	56.000000	6.000000	6.100000	146.000000	1.700000	2025-01-01 00:00:00
25%	28.556200	77.100000	19.500000	44.000000	977.500000	4.000000	152.000000	51.100000	80.600000	464.000000	14.900000	2025-04-02 05:45:00
50%	28.609300	77.160100	26.500000	65.000000	983.400000	6.000000	170.000000	76.400000	151.800000	664.000000	26.900000	2025-07-02 11:30:00
75%	28.646900	77.261800	30.500000	82.000000	988.900000	8.700000	238.000000	110.400000	325.000000	1078.000000	47.100000	2025-10-01 17:15:00
max	28.704100	77.316000	43.800000	100.000000	998.800000	27.100000	2742.000000	443.300000	3263.400000	7432.000000	220.100000	2025-12-31 23:00:00
std	0.058927	0.096519	7.618231	23.454666	6.688582	3.478066	327.979524	55.363812	412.782899	600.845140	28.978905	NaN

4. See if any null values are present. Display count of null values.

Program

```
import numpy as np
import pandas as pd

df = pd.read_csv('delhi-weather-aqi-2025.csv')
null_counts = df.isnull().sum()
print(null_counts)
print("Total missing:", null_counts.sum())
```

Output

```
...  date_ist      0
     time_ist      0
     location      0
     lat          0
     lon          0
     temp_c        0
     humidity      0
     pressure_mb    0
     windspeed_kph  0
     condition_text 0
     description    0
     aqi_index      0
     pm2_5          0
     pm10           0
     co             0
     no2            0
     dtype: int64
     Total missing: 0
```

5. Choose appropriate technique to handle missing values.(imputation with use of inplace)

Program

```
np.random.seed(42)
df_missing = df.copy()
rows_t = np.random.choice(df_missing.index, 150, replace=False)
rows_a = np.random.choice(df_missing.index, 100, replace=False)
df_missing.loc[rows_t, 'temp_c'] = np.nan
df_missing.loc[rows_a, 'aqi_index'] = np.nan

temp_mean = df_missing['temp_c'].mean()
aqi_mean = df_missing['aqi_index'].mean()
df_missing.fillna({'temp_c': temp_mean, 'aqi_index': aqi_mean},
inplace=True)

print(df_missing.isnull().sum()[['temp_c', 'aqi_index']])
```

output

```
... temp_c      0
   aqi_index    0
   dtype: int64
```

6. Use sorting of data in dataframe to display topmost 5 or 8 records based on one or more column values(conditional filtering)

Program

```
top8_hottest = df.sort_values(by='temp_c', ascending=False).head(8)
print(top8_hottest[['date_ist', 'time_ist', 'location', 'temp_c']])

anand_vihar = df[df['location'] == 'Anand Vihar']
top5_polluted_av = anand_vihar.sort_values(by='aqi_index',
ascending=False).head(5)
print(top5_polluted_av[['date_ist', 'time_ist', 'aqi_index',
'pm2_5']])
```

Output

...	date_ist	time_ist	location	temp_c
23271	11/06/2025	15:00	Okhla Phase III	43.8
23438	12/06/2025	14:00	Rohini	43.8
22982	09/06/2025	14:00	Okhla Phase III	43.8
22934	09/06/2025	14:00	Connaught Place	43.8
22983	09/06/2025	15:00	Okhla Phase III	43.7
23270	11/06/2025	14:00	Okhla Phase III	43.7
22910	09/06/2025	14:00	Anand Vihar	43.5
22911	09/06/2025	15:00	Anand Vihar	43.5
	date_ist	time_ist	aqi_index	pm2_5
9219	06/03/2025	3:00	830	116.4
9218	06/03/2025	2:00	829	122.5
9220	06/03/2025	4:00	829	109.9
9217	06/03/2025	1:00	827	125.7
9221	06/03/2025	5:00	827	105.9



7. Get frequency listing of any one relevant column(2 cases)

Program

```
print(df['location'].value_counts())  
print(df['condition_text'].value_counts())
```

Output

```
... location
Anand Vihar      8760
Connaught Place  8760
Dwarka           8760
Okhla Phase III  8760
Rohini           8760
IGI Airport      8760
Name: count, dtype: int64
condition_text
Clear sky        30086
Overcast         10285
Mainly clear     4444
Drizzle: Light   3090
Partly cloudy    2557
Drizzle: Moderate 643
Rain: Moderate   421
Rain: Slight     416
Fog              334
Drizzle: Dense   221
Rain: Heavy      63
Name: count, dtype: int64
```

8. Sorting of rows and columns,(implicit and explicit indexing)

Program

```
print(df.sort_index(ascending=False).head(3)[['location', 'temp_c']])

df_explicit = df.set_index('datetime')
print(df_explicit.sort_index().head(3)[['location', 'temp_c']])
```

Output

...	location	temp_c
52559	IGI Airport	13.3
52558	IGI Airport	13.4
52557	IGI Airport	13.8
	location	temp_c
datetime		
2025-01-01	Anand Vihar	8.1
2025-01-01	Okhla Phase III	8.6
2025-01-01	Dwarka	8.0

9. Row access with compound conditions (3 cases)

Program

```
case1 = df.loc[(df['temp_c'] > 40) & (df['location'] == 'Anand Vihar'),  
               ['date_ist', 'time_ist', 'temp_c']]  
  
case2 = df.loc[(df['aqi_index'] > 300) & (df['pm2_5'] > 200),  
               ['location', 'aqi_index', 'pm2_5']]  
  
case3 = df.loc[(df['humidity'] < 20) & (df['windspeed_kph'] > 10),  
               ['date_ist', 'location', 'humidity', 'windspeed_kph']]  
  
print(case1.head()); print(case2.head()); print(case3.head())
```

Output

```
...      date_ist  time_ist  temp_c
14126  09/04/2025   14:00    40.2
14127  09/04/2025   15:00    40.3
15277  17/04/2025   13:00    40.2
15278  17/04/2025   14:00    40.3
15279  17/04/2025   15:00    40.5
      location  aqi_index  pm2_5
1313  Anand Vihar      315  231.9
1314  Anand Vihar      317  262.6
1315  Anand Vihar      318  282.2
1316  Anand Vihar      318  293.4
1317  Anand Vihar      317  293.4
      date_ist      location  humidity  windspeed_kph
9517  08/03/2025    Anand Vihar      19         12.6
9518  08/03/2025    Anand Vihar      18         12.5
9588  08/03/2025  Okhla Phase III      19         12.0
9589  08/03/2025  Okhla Phase III      18         12.4
9590  08/03/2025  Okhla Phase III      18         12.1
```

10. Min/max analysis

Program

```
print("Max:", df['temp_c'].max(), "Min:", df['temp_c'].min())
print(df.loc[df['temp_c'].idxmax()])
print("Max AQI:", df['aqi_index'].max(), "Min AQI:",
df['aqi_index'].min())
```

Output

```
... Max: 43.8 Min: 5.7
date_ist          09/06/2025
time_ist          14:00
location          Connaught Place
lat               28.6304
lon               77.2177
temp_c            43.8
humidity          16
pressure_mb       975.5
windspeed_kph     13.1
condition_text    Clear sky
description        WMO Code 0
aqi_index         537
pm2_5             75.0
pm10              499.2
co                296
no2               3.7
datetime          2025-06-09 14:00:00
Name: 22934, dtype: object
Max AQI: 2742 Min AQI: 56
```

11. Groupby (2 cases)

Program

```
avg_temp_by_location =
df.groupby('location')['temp_c'].mean().sort_values(ascending=False)
print(avg_temp_by_location)

df['month'] = df['datetime'].dt.month_name()
avg_aqi_loc_month = df.groupby(['location', 'month'])['aqi_index'].mean()
print(avg_aqi_loc_month.head(6))
```

Output

```
*** location
Connaught Place      25.376336
Anand Vihar          25.308607
Okhla Phase III      24.986473
IGI Airport          24.841336
Dwarka               24.833356
Rohini               24.800331
Name: temp_c, dtype: float64
location  month
Anand Vihar April      240.105556
           August     179.489247
           December   193.161290
           February    153.620536
           January     189.924731
           July        147.389785
Name: aqi_index, dtype: float64
```

12. New column from existing columns

Program

```
df['heatwave_alert'] = np.where(df['temp_c'] >= 40, 'Yes', 'No')
print(df['heatwave_alert'].value_counts())
```

Output

```
*** heatwave_alert
No      51744
Yes       816
Name: count, dtype: int64
```

13. Aggregate functions with groupby (2 cases)

Program

```
agg1 = df.groupby('location').agg({'temp_c': 'mean', 'aqi_index': 'max'})
print(agg1)

agg2 = df.groupby('month').agg({'temp_c': ['mean', 'max', 'min']})
print(agg2.head(4))
```

Output

...	temp_c	aqi_index
location		
Anand Vihar	25.308607	830
Connaught Place	25.376336	830
Dwarka	24.833356	2742
IGI Airport	24.841336	2742
Okhla Phase III	24.986473	2742
Rohini	24.800331	830
	temp_c	
	mean	max
month		min
April	31.107106	42.5
August	28.778898	34.7
December	15.527285	25.9
February	18.552257	30.8

14. Selecting a particular group

Program

```
grouped = df.groupby('location')
anand_vihar_group = grouped.get_group('Anand Vihar')
print(anand_vihar_group.shape)
```

Output

```
... (8760, 19)
```

15. Correlation

Program

```
print(df['temp_c'].corr(df['aqi_index']))
print(df[['temp_c', 'humidity', 'aqi_index', 'pm2_5', 'pm10']].corr())
```

Output

```
... 0.19221972508440102
      temp_c  humidity  aqi_index  pm2_5  pm10
temp_c      1.000000 -0.511204   0.192220 -0.304957  0.266424
humidity    -0.511204  1.000000  -0.330126  0.036464 -0.380884
aqi_index    0.192220 -0.330126   1.000000  0.524649  0.831292
pm2_5       -0.304957  0.036464   0.524649  1.000000  0.615478
pm10         0.266424 -0.380884   0.831292  0.615478  1.000000
```

16. Normalization

Program

```
df['temp_c_normalized'] = (df['temp_c'] - df['temp_c'].min()) /
(df['temp_c'].max() - df['temp_c'].min())
print(df[['temp_c', 'temp_c_normalized']].head())

df['aqi_zscore'] = (df['aqi_index'] - df['aqi_index'].mean()) /
df['aqi_index'].std()
```

Output

```
...      temp_c  temp_c_normalized
0         8.1         0.062992
1         7.7         0.052493
2         7.5         0.047244
3         7.8         0.055118
4         7.3         0.041995
```

17. Merge, concat, join

Program

```
weather_part = df[['date_ist', 'time_ist', 'location', 'temp_c',
'humidity']]
pollution_part = df[['date_ist', 'time_ist', 'location', 'aqi_index',
'pm2_5', 'pm10']]

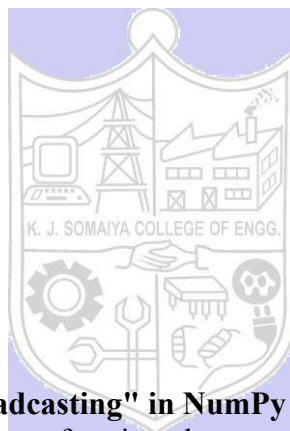
merged_df = pd.merge(weather_part, pollution_part, on=['date_ist',
'time_ist', 'location'], how='inner')
print(merged_df.shape)

concatenated_df = pd.concat([df.iloc[:100], df.iloc[100:200]], axis=0)
print(concatenated_df.shape)
```

```
joined_df = weather_part.set_index(['date_ist', 'time_ist',
'location']).join(
    pollution_part.set_index(['date_ist', 'time_ist',
'location']))
print(joined_df.shape)
```

Output

```
... (52560, 8)
(200, 21)
(52560, 5)
```



Post-Lab Questions:

1. Explain the concept of "Broadcasting" in NumPy with the help of suitable example

Ans. Broadcasting is NumPy's rule for performing element-wise operations on arrays of different shapes without writing explicit loops or manually replicating data. When shapes differ, NumPy 'stretches' the smaller array across the larger one along dimensions of size 1, provided the shapes are compatible (equal, or one of them is 1). Example: `import numpy as np arr = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2,3) scalar = 10 print(arr + scalar) # [[11 12 13] # [14 15 16]]` Here the scalar 10 (conceptually shape (1,)) is broadcast across every element of the (2,3) array, so it is added to each entry as if it had been expanded to match arr's shape - without NumPy actually copying the value in memory. The same rule is what lets us do `df_clean['ACR_normalized'] = (df_clean['ACR'] - min_v) / (max_v - min_v)` in the normalization step above: a single scalar min/max is broadcast across the whole ACR column in one vectorised operation.

2. Explain the difference between label-based indexing (`df.loc[]`) and integer-based indexing (`df.iloc[]`) in Pandas with the help of suitable example.

Ans. `df.loc[]` is label-based indexing — you pass the actual index labels (or column names) you want. `df.iloc[]` is integer-position-based indexing — you pass 0-based positional integers regardless of what the labels actually are.

Example:

```
df_indexed = df.set_index('datetime')
df_indexed.loc['2025-01-01 00:00:00'] # finds the row with THIS exact label
# location Anand Vihar
# temp_c 8.1
```

```
df_indexed.iloc[2] # returns whatever row is in position 2 (3rd row)
# location Okhla Phase III
```

```
# temp_c    8.6
```

loc[...] finds the row whose label literally matches, wherever it sits in the dataframe. iloc[2] instead returns whichever row currently happens to be in position 2 — this would point to a different row entirely if the dataframe were re-sorted or filtered beforehand, which is why loc is safer once a meaningful index has been set.

Course Outcomes:

Ability to apply data handling techniques (reading, cleaning, transforming, aggregating and merging real-world data) using Python's NumPy and Pandas libraries to derive meaningful insight from a raw dataset.

Conclusion: In this experiment, the Delhi 2025 hourly weather and air-quality dataset (52,560 records, 16 variables) was used to practice the full range of Pandas/NumPy data-handling operations: reading and inspecting data, checking and filling missing values, sorting and conditional filtering, frequency counts, label- vs position-based indexing, groupby and aggregation, derived-column creation, correlation analysis, min-max normalization, and merging/joining/concatenation. The exercise showed that real heatwave hours are rare (only about 1.6% of the year crossed 40°C) and tend to occur together in clusters rather than as isolated spikes, while air-quality swings much more often and independently of temperature. Working through each technique on this dataset built practical confidence in choosing the right Pandas/NumPy tool for a given data-handling task.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:**Books/ Journals/ Websites:**

1. S.C. Gupta, V. K. Kapoor, *Fundamentals of Mathematical Statistics*, Sultan Chand and Sons, 2020.
2. P. N. Tan, M. Steinbach, Vipin Kumar, *Introduction to Data Mining*, Pearson Education, 2016.
3. Han, Kamber, *Data Mining Concepts and Techniques*, Morgan Kaufmann, 3rd Edition, 2012
4. NumPy Documentation - <https://numpy.org/doc/>
5. Pandas Documentation - <https://pandas.pydata.org/>